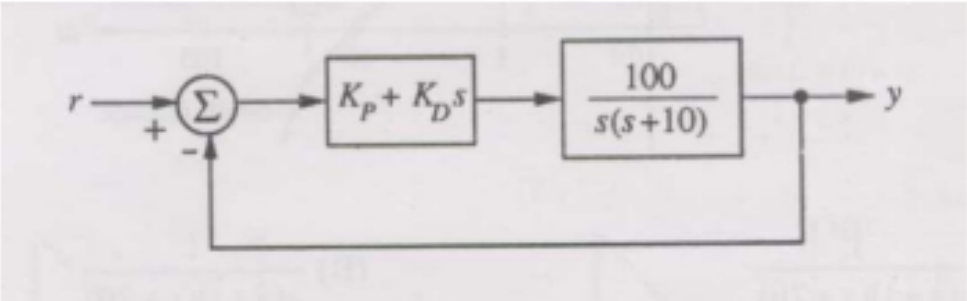
Control Systems Problem

Darsh Pankaj Gajare — EE25BTECH11020

Question

A control system with a PD controller is shown. If the velocity error constant $K_v = 1000$ and the damping ratio $\zeta = 0.5$, then the values of K_P and K_D are:



1. $K_P = 100,\ K_D = 0.09$
2. $K_P = 100,\ K_D = 0.9$
3. $K_P = 10,\ K_D = 0.09$
4. $K_P = 10,\ K_D = 0.9$

Given

- Controller: $K_p + K_D s$
- Plant: $\dfrac{100}{s(s+10)}$
- Unity feedback system
- $K_v = 1000$
- $\zeta = 0.5$

Find:

$$K_p, \quad K_D$$

Step 1: Open-loop Transfer Function

$$G(s) = \frac{100(K_p + K_D s)}{s(s+10)}$$

Step 2: Velocity Error Constant

$$K_v = \lim_{s \to 0} sG(s)$$

$$K_v = \lim_{s \to 0} \frac{100(K_p + K_D s)}{s + 10}$$

As $s \to 0$:

$$K_v = \frac{100K_p}{10} = 10K_p$$

$$10K_p = 1000 \Rightarrow K_p = 100$$

Step 3: Characteristic Equation

$$1 + G(s) = 0$$

$$s(s + 10) + 100(K_p + K_D s) = 0$$

$$s^2 + (10 + 100K_D)s + 100K_p = 0$$

Step 4: Standard Form Comparison

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$

Matching terms:

$$\omega_n^2 = 100K_p$$

$$2\zeta\omega_n = 10 + 100K_D$$

Step 5: Natural Frequency

$$\omega_n^2 = 100 \times 100 = 10000$$

$$\omega_n = 100$$

Step 6: Damping Ratio

$$2\zeta\omega_n = 2 \times 0.5 \times 100 = 100$$

$$10 + 100K_D = 100$$

$$100K_D = 90 \Rightarrow K_D = 0.9$$

Final Answer

Result

$$K_p = 100, \quad K_D = 0.9$$

```python
import numpy as np
import control as ctrl

# ------------------------------
# Given solution
# ------------------------------
Kp = 100
Kd = 0.9

# ------------------------------
# Define open-loop system
# G(s) = 100(Kp + Kd s) / (s(s+10))
# ------------------------------
num = [100*Kd, 100*Kp]
den = [1, 10, 0]

G = ctrl.TransferFunction(num, den)

# ------------------------------
# 1. Verify Kv
# Kv = lim s->0 [sG(s)]
```

```python
# ------------------------------
Kv = ctrl.dcgain(ctrl.TransferFunction([1, 0], [1]) *
    G)

print("Computed Kv =", Kv)

# ------------------------------
# 2. Closed-loop system
# ------------------------------
T = ctrl.feedback(G, 1)

# ------------------------------
# 3. Extract poles
# ------------------------------
poles = ctrl.poles(T)
print("Closed-loop poles:", poles)

# ------------------------------
# 4. Compute damping ratio
# ------------------------------
dominant_pole = poles[0]  # complex pole
```

```python
zeta = -np.real(dominant_pole) / np.abs(dominant_pole)

print("Computed damping ratio =", zeta)

# ------------------------------
# 5. Natural frequency
# ------------------------------
wn = np.abs(dominant_pole)
print("Natural frequency =", wn)
```

Velocity Error Constant (K_v)

Definition

$$K_v = \lim_{s \to 0} sG(s)$$

Meaning

K_v measures how well a system tracks a **ramp input** (continuously increasing input).

Steady-State Error

$$e_{ss} = \frac{1}{K_v}$$

Practical Interpretation

- Higher $K_v \Rightarrow$ smaller tracking error
- Lower $K_v \Rightarrow$ larger tracking error

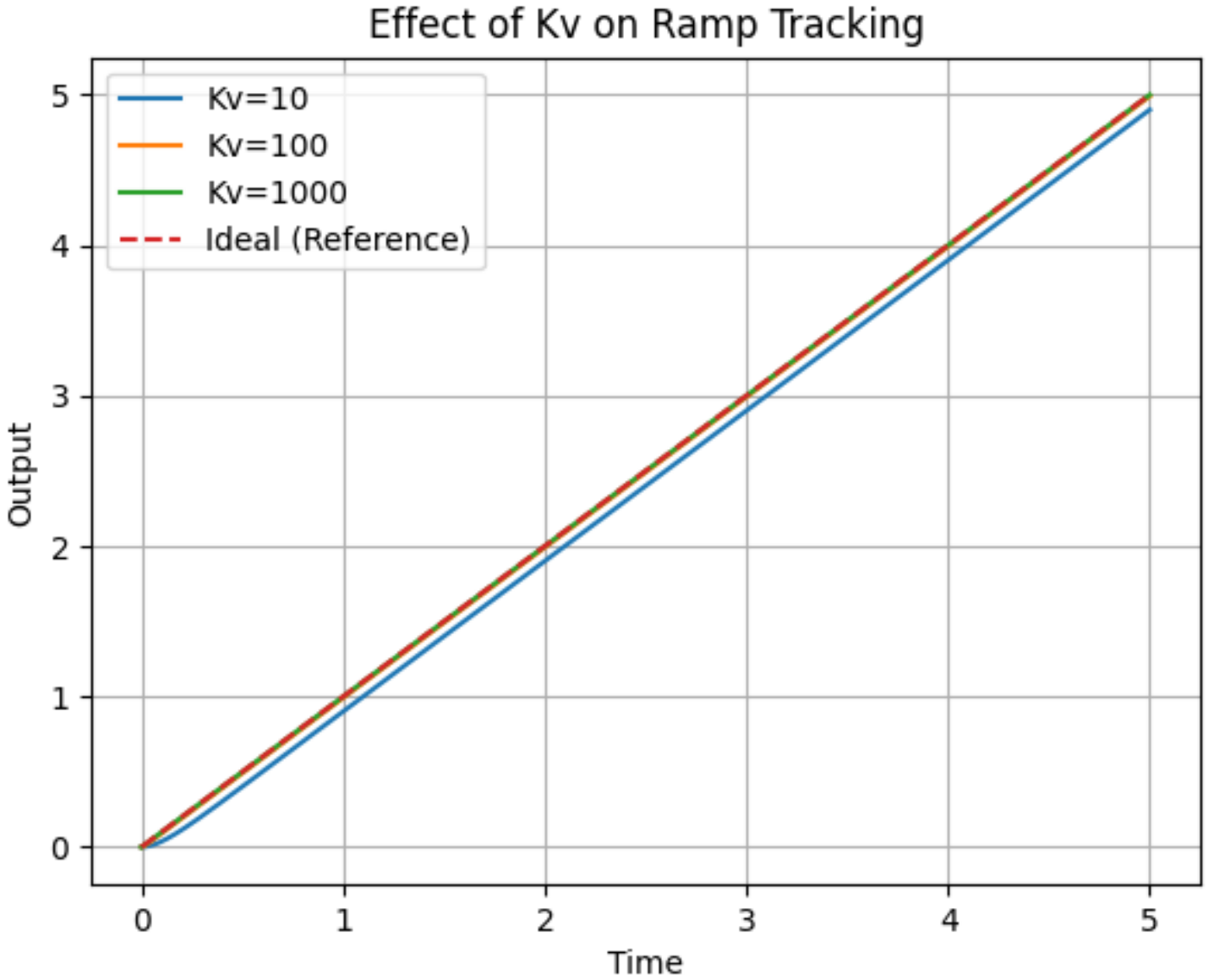
Velocity Error Constant (K_v)

Example

Cruise control:

- High $K_v \Rightarrow$ car follows increasing speed closely
- Low $K_v \Rightarrow$ car falls behind desired speed

Ramp Tracking Demonstration



Damping Ratio (ζ)

Definition

$$\zeta = \text{Damping Ratio}$$

Standard Form

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$

Meaning

ζ determines how **smooth or oscillatory** the system response is.

Behavior

- $0 < \zeta < 1 \Rightarrow$ Oscillatory (underdamped)
- $\zeta = 0.5 \Rightarrow$ Moderate overshoot
- $\zeta = 1 \Rightarrow$ No oscillation (critical damping)
- $\zeta > 1 \Rightarrow$ Slow response (overdamped)

Damping Ratio (ζ)

Practical Meaning

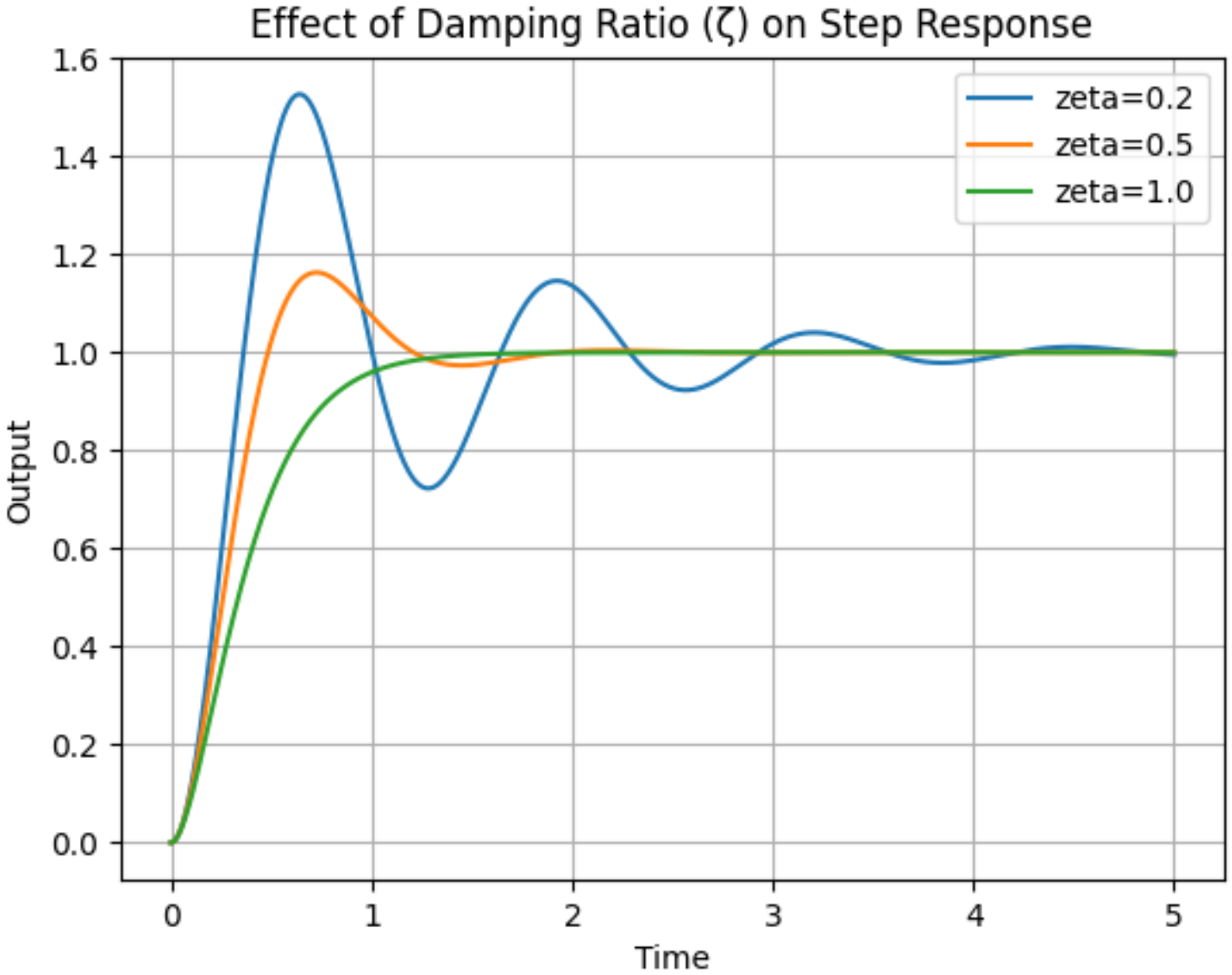
ζ determines how **smoothly the system reaches its target**.

Example

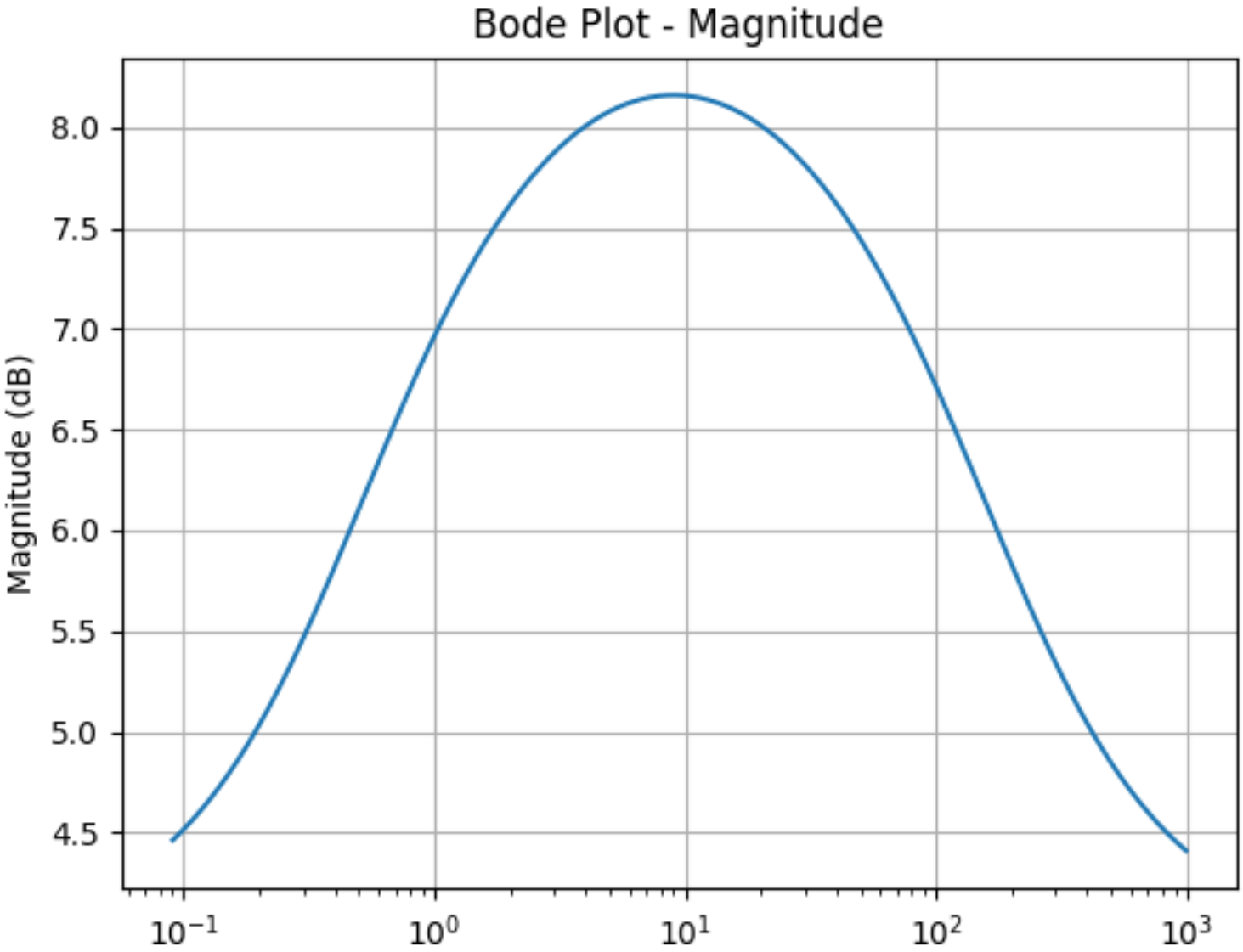
Car suspension:

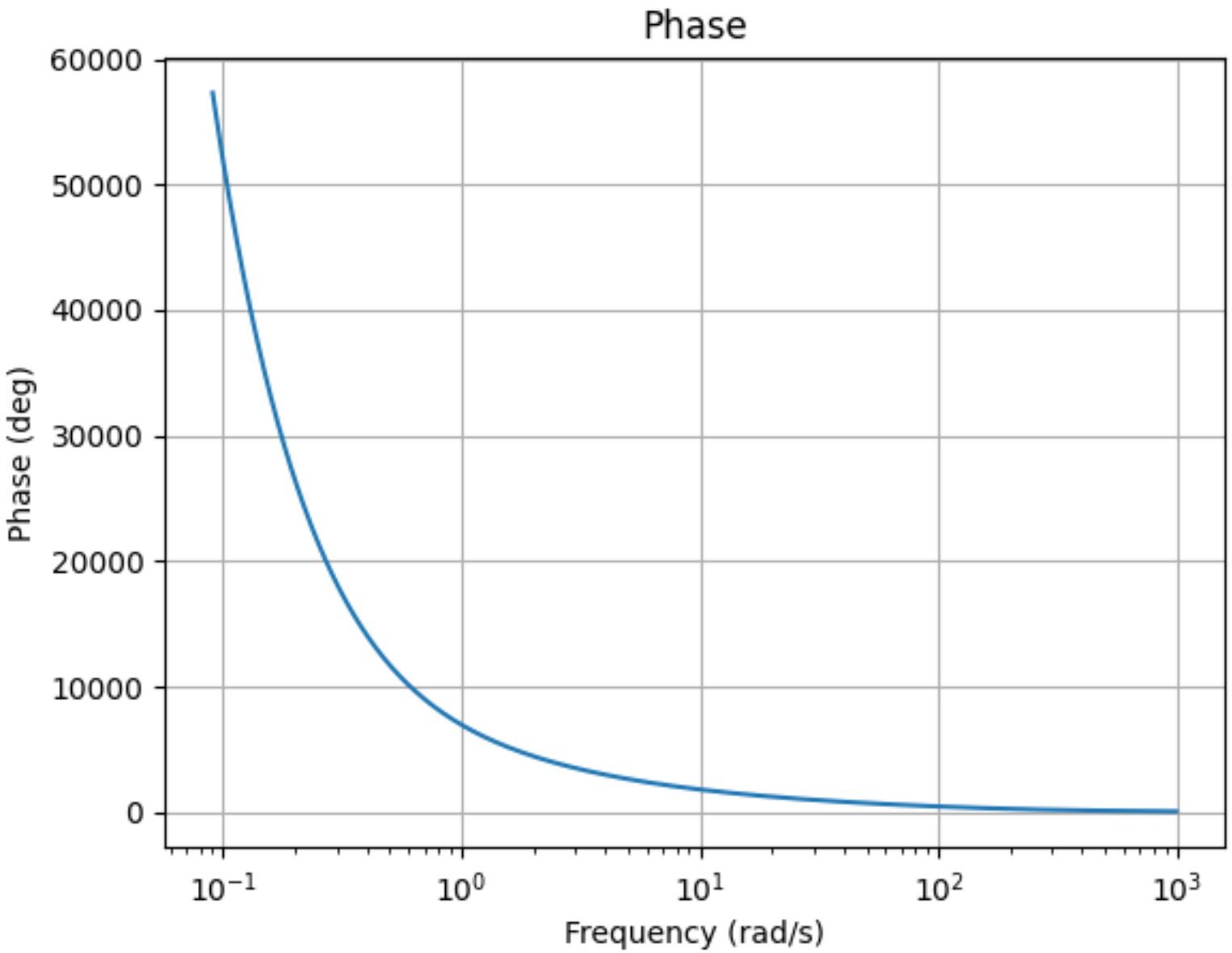
- Low $\zeta \Rightarrow$ bouncing
- Proper $\zeta \Rightarrow$ smooth settling

Damping Demonstration



Bode Plots of Transfer Function




Phase
60000
50000
40000
30000
20000
10000
0
Phase (deg)
10^{-1}
10^{0}
10^{1}
10^{2}
10^{3}
Frequency (rad/s)